

ElastiCache Redis

Cache-Aside · Sessions · Rate Limiting · Pub/Sub · HA · Cluster Mode

AWS Tutorial · ShopFlow

08-00

Chapter Overview — ElastiCache Redis at ShopFlow

ShopFlow uses ElastiCache Redis as the primary caching layer between ECS services and Aurora PostgreSQL. Redis eliminates repeated database reads for hot data (product catalog, session tokens, rate limiting counters) and provides sub-millisecond response times at 1M MAU.

Cache Use-Case	Data Cached	TTL	Cache Strategy	Hit Rate Target
Product catalog	Product details, images, prices	1 hour	Cache-aside (lazy load)	95% — catalog changes infrequently
Session tokens	JWT validation result + user roles	15 min	Write-through on login	99% — every API request validates
Homepage feed	Top 20 products by category	5 min	Read-through via Lambda	90% — refreshed on inventory change
Rate limiting	API call counters per IP + endpoint	1 min window	Write-around (Redis as source of truth)	N/A — counters, not cache
Search autocomplete	Top-10 product name completions	30 min	Cache-aside per query prefix	80% — long tail of unique queries
Cart summary	Item count badge, cart total	30 sec	Write-through on cart update	85% — users browse without editing

Metric	Normal Peak	Black Friday Peak	Redis Config
Requests/s to Redis	5,000 ops/s	50,000 ops/s	cache.r7g.large (26GB RAM, 3-node cluster)
Cache hit rate (product)	95%	97% (repeated browsing)	1hr TTL, LRU eviction
Memory used	8 GB (50M cached objects)	12 GB (Black Friday catalog)	26 GB capacity — 54% headroom on normal
Latency (p99)	<0.5ms	<1ms	In-region: us-east-1 ECS → Redis <0.1ms
Replication	1 primary + 2 replicas (Multi-AZ)	Same — auto failover in <60s	reader endpoint distributes reads

■ ElastiCache Redis cluster-mode disabled (replication group): 1 shard, 1 primary + 2 replica nodes. All three nodes hold the full dataset. Primary handles all writes; reader endpoint load-balances reads across replicas. ShopFlow uses this simpler topology — cluster-mode (sharding across multiple primaries) only needed when dataset > 26 GB or write throughput exceeds 100K ops/s.

08-01

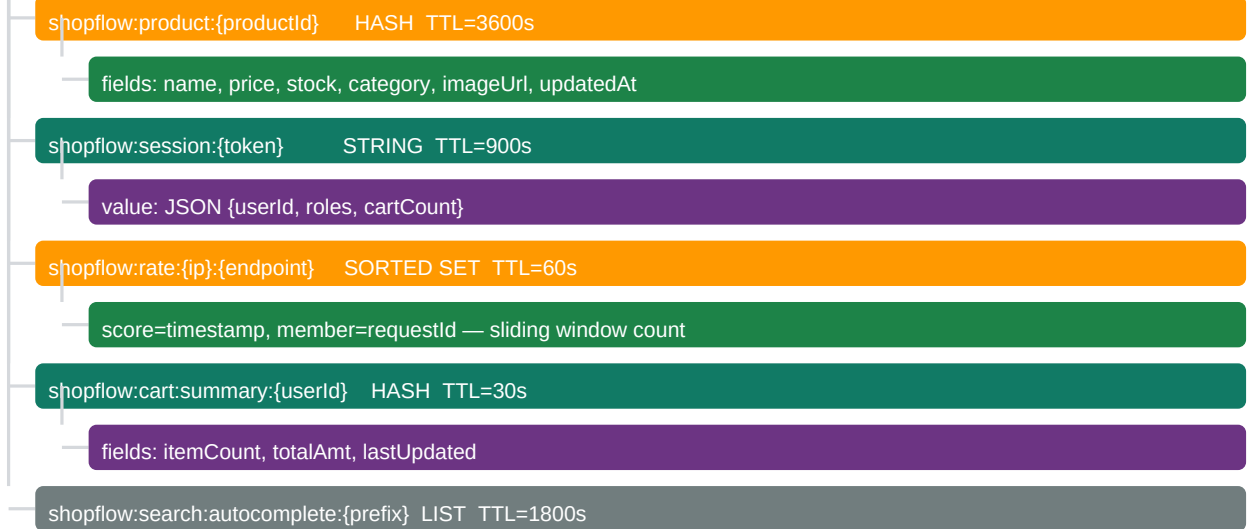
Redis Data Structures & ShopFlow Mappings

Redis provides five core data structures plus HyperLogLog, Streams, and probabilistic structures. ShopFlow maps each use-case to the optimal structure — using HASH for product objects, SORTED SET for leaderboards and rate limiting, and STRING for session tokens.

Redis Type	Command Examples	ShopFlow Use-Case	Why This Type
STRING	SET, GET, INCR, EXPIRE	Session tokens, JWT validation cache	Simplest O(1) get/set; supports atomic INCR for counters
HASH	HSET, HGET, HGETALL, HMSET	Product details (id, name, price, stock)	Single key for one product; HGET individual fields vs multiple keys
LIST	LPUSH, RPOP, LRANGE	Recent order IDs per user (last 10)	O(1) push/pop; LRANGE 0 9 for last 10 without scan
SORTED SET	ZADD, ZRANGE, ZREVRANGEBYSCORE	Rate limiting (score=timestamp), trending products by score	Ordering by score; O(log N) add/remove; atomic range queries
SET	SADD, SMEMBERS, SISMEMBER	Product IDs in a category (dedup), user permissions	O(1) membership check; set intersection for multi-filter
HyperLogLog	PFADD, PFCOUNT	Unique daily visitors per product page	Fixed 12KB memory, ~0.81% error regardless of cardinality (vs O(N))

Redis Type	Command Examples	ShopFlow Use-Case	Why This Type
Streams	XADD, XREAD, XACK	Real-time order event log, dashboard	Persistent, consumer group ACK, replay from any c

ShopFlow Redis Key Namespace Design



■ Use colon (:) as a Redis key namespace separator. This is a convention, not enforced by Redis — but it enables Redis Insight and redis-cli --pattern "shopflow:product:*" to group keys visually. Include entity type, entity ID, and optional sub-key. Avoid very long keys (>100 bytes) — key lookups are O(1) on value but key memory overhead is significant at 50M+ cached objects.

08-02

ElastiCache Cluster CDK — Multi-AZ Replication Group

ShopFlow deploys ElastiCache Redis as a Replication Group (cluster-mode disabled): one shard with one primary and two read replicas in separate AZs. CDK provisions the subnet group, security group, parameter group, and CloudWatch alarms as a single stack.

ElastiCache CDK

```
const redisSubnetGroup = new elasticache.CfnSubnetGroup(this, "RedisSubnets", {
  description: "ShopFlow Redis data-tier subnets",
  subnetIds: vpc.selectSubnets({ subnetType: SubnetType.PRIVATE_ISOLATED }).subnetIds,
  cacheSubnetGroupName: "shopflow-redis-subnets",
});

const sgRedis = new ec2.SecurityGroup(this, "RedisSecurityGroup", {
  vpc, description: "ElastiCache Redis – restrict to app tier",
  allowAllOutbound: false,
});

sgRedis.addIngressRule(sgEcs, Port.tcp(6379), "ECS tasks -> Redis");

const redisParamGroup = new elasticache.CfnParameterGroup(this, "RedisParams", {
  cacheParameterGroupFamily: "redis7",
  description: "ShopFlow Redis 7 parameters",
  properties: {
    "maxmemory-policy": "allkeys-lru", // evict LRU on memory pressure
    "lazyfree-lazy-eviction": "yes", // async eviction (non-blocking)
    "activeremhashing": "yes", // incremental hash table resize
    "timeout": "300", // idle connection timeout 5min
    "tcp-keepalive": "60",
  },
});

const redis = new elasticache.CfnReplicationGroup(this, "ShopFlowRedis", {
  replicationGroupDescription: "ShopFlow session + catalog cache",
```

```

cacheNodeType: "cache.r7g.large",
engine: "redis",
engineVersion: "7.1",
numCacheClusters: 3, // 1 primary + 2 replicas
multiAzEnabled: true,
automaticFailoverEnabled: true,
cacheSubnetGroupName: redisSubnetGroup.ref,
cacheParameterGroupName: redisParamGroup.ref,
securityGroupIds: [sgRedis.securityGroupId],
atRestEncryptionEnabled: true,
transitEncryptionEnabled: true,
authToken: redisAuthSecret.secretValue.unsafeUnwrap(),
snapshotRetentionLimit: 7,
snapshotWindow: "03:00-04:00",
});

```

■ **cache.r7g.large** (Graviton3): 26.32 GB RAM, 2 vCPU, up to 12.5 Gbps network. Compared to r6g.large (26.32 GB RAM, 2 vCPU): Graviton3 is ~20% higher throughput at same price. Use Graviton (r7g, m7g) for all new ElastiCache deployments — same pricing tiers as x86.

08-03

Spring Boot Lettuce Client — Connection Pool & Config

ShopFlow uses Spring Data Redis with Lettuce as the client. Lettuce uses a single connection per thread-model (non-blocking NIO) by default — connection pooling is optional but recommended for blocking commands. Configuration uses `application.yml` and a custom `LettuceConnectionFactory`.

application.yml — Redis Config

```

spring:
  data:
    redis:
      host: ${REDIS_PRIMARY_ENDPOINT} # injected from CDK output
      port: 6379
      password: ${REDIS_AUTH_TOKEN} # from Secrets Manager
      ssl: true
      timeout: 2000ms # command timeout
      lettuce:
        pool:
          max-active: 20 # max connections
          max-idle: 10 # idle pool size
          min-idle: 5 # min warm connections
          max-wait: 500ms # wait time on pool exhaustion → fail fast
        cluster:
          refresh:
            adaptive: true # update topology on MOVED/ASK responses

```

RedisConfig.java — Custom Factory with Read-From-Replica

```

@Configuration
public class RedisConfig {
    @Bean
    public LettuceConnectionFactory redisConnectionFactory(
        @Value("${spring.data.redis.host}") String primary,
        @Value("${REDIS_READER_ENDPOINT}") String reader) {
        // Read from replica, write to primary
        LettuceClientConfiguration clientConfig = LettuceClientConfiguration.builder()
            .readFrom(ReadFrom.REPLICA_PREFERRED) // reads go to replica when available
            .commandTimeout(Duration.ofMillis(2000))
            .useSsl()
            .build();
        RedisStandaloneConfiguration redisConfig =
            new RedisStandaloneConfiguration(primary, 6379);

```

```
redisConfig.setPassword(RedisPassword.of(System.getenv("REDIS_AUTH_TOKEN")));
return new LettuceConnectionFactory(redisConfig, clientConfig);
}

@Bean
public RedisTemplate<String, Object> redisTemplate(
    LettuceConnectionFactory factory) {
    RedisTemplate<String, Object> t = new RedisTemplate<>();
    t.setConnectionFactory(factory);
    t.setKeySerializer(new StringRedisSerializer());
    t.setValueSerializer(new GenericJackson2JsonRedisSerializer());
    return t;
}
}
```

■ Never use the default `JdkSerializationRedisSerializer` — it stores Java class names in the value, breaking if the class is renamed or removed. Always use `GenericJackson2JsonRedisSerializer` for objects or `StringRedisSerializer` for strings. This also makes values human-readable in `redis-cli` for debugging.

08-04

Cache-Aside Pattern — Product Catalog

Cache-aside (lazy load): application checks Redis first; on miss, loads from Aurora, writes to Redis with TTL. On write/update, application invalidates or updates the cache key. ShopFlow implements this pattern for product catalog with `@Cacheable` and manual `RedisTemplate` calls.

ProductService.java — Cache-Aside with @Cacheable

```
@Service
public class ProductService {
    private final ProductRepository productRepo;
    private final RedisTemplate<String, Object> redis;
    private static final String PFX = "shopflow:product:";
    private static final long TTL_SEC = 3600; // 1 hour
    // GET — cache-aside: try Redis, fall through to Aurora
    @Cacheable(value="products", key="#productId",
        unless="#result == null")
    public ProductDto getProduct(String productId) {
        log.debug("Cache MISS for product {}", productId);
        return productRepo.findById(productId) // Aurora read
            .map(ProductDto::fromEntity)
            .orElse(null);
    }

    // UPDATE — write-through: update DB, then update cache
    @CacheEvict(value="products", key="#dto.productId")
    @Transactional
    public ProductDto updateProduct(ProductDto dto) {
        Product entity = productRepo.save(dto.toEntity()); // Aurora write
        // Cache will be rebuilt on next GET (lazy population)
        // OR manually repopulate (write-through):
        String key = PFX + dto.getProductId();
        redis.opsForHash().putAll(key, dto.toHashMap());
        redis.expire(key, Duration.ofSeconds(TTL_SEC));
        return ProductDto.fromEntity(entity);
    }
}
```

Cache Strategy	Write Path	Read Path	Consistency	Best For
Cache-aside (lazy)	Write DB, evict cache key	Check cache, miss → read DB, populate cache	Eventual — first read after write	Frequently read data: products, categories
Write-through	Write DB + update cache atomically	Check cache, always fresh	Strong — cache always fresh	Frequently read data: session tokens
Write-behind (async)	Write cache, async flush to DB	Check cache	Strong reads, async writes	Write-heavy: analytics counters, metrics

Cache Strategy	Write Path	Read Path	Consistency	Best For
Read-through	Cache loads from DB on miss	Cache manages DB read automatically	Eventual — first reader wins	External cache providers; not Spring

■ @Cacheable uses Spring Cache abstraction backed by Redis. The cache name "products" maps to Redis key "products:::{productId}". Unless clause prevents caching null (product not found). @CacheEvict removes the cached entry after update. For bulk eviction (clear entire category after import), use @CacheEvict(allEntries=true) or redisTemplate.delete(keys).

08-05 Session Caching — Write-Through + JWT Validation

ShopFlow caches JWT validation results in Redis to avoid verifying the cryptographic signature on every API request. On login, the JWT validation result (userId, roles, permissions) is stored in Redis with the token expiry as TTL. Auth middleware fetches from Redis first.

SessionCacheService.java — Write-Through Session Cache

```
@Service
public class SessionCacheService {
    private final RedisTemplate<String,String> redis;
    private final ObjectMapper json;
    private static final String PFX = "shopflow:session:";
    // Called on successful login – write-through to Redis
    public void cacheSession(String jwtToken, SessionPrincipal principal,
    Duration ttl) {
        String key = PFX + hashToken(jwtToken); // SHA-256 hash – never raw token as key
        String value = json.writeValueAsString(principal);
        redis.opsForValue().set(key, value, ttl); // STRING: SET key value EX ttl
    }
    // Called in auth filter on EVERY API request
    public Optional<SessionPrincipal> getSession(String jwtToken) {
        String key = PFX + hashToken(jwtToken);
        String raw = redis.opsForValue().get(key); // 0(1) – sub-ms
        if (raw == null) return Optional.empty(); // Cache miss → full JWT verify
        return Optional.of(json.readValue(raw, SessionPrincipal.class));
    }
    // Called on logout or password change – immediate invalidation
    public void invalidateSession(String jwtToken) {
        redis.delete(PFX + hashToken(jwtToken));
    }
}
```

Auth Flow Step	Without Redis Cache	With Redis Session Cache	Latency Saved
Token arrival at API Gateway	—	—	—
JWT signature verify (crypto)	~5ms (RSA-256 verify)	Skip if Redis hit	~5ms saved (99% requests)
DB lookup: user + roles	~3ms (Aurora)	Skip if Redis hit	~3ms saved
Permission check	~1ms in-memory	Cached in Redis value	~0ms (data already fetched)
Total per authenticated request	~9ms auth overhead	~0.3ms Redis GET	~8.7ms saved × 1,000 req/s = 8.7s la

■ Never store the raw JWT token as the Redis key — tokens are 200-500 byte strings visible in redis-cli, logs, and memory profilers. Always hash the token (SHA-256, 64-char hex). Store only the validation result (userId, roles), not the full JWT. Set the Redis TTL equal to the JWT expiry — stale sessions must not be served after token expiry.

08-06 Rate Limiting — Redis SORTED SET Sliding Window

ShopFlow implements API rate limiting using Redis Sorted Sets for a sliding window algorithm. Each request is added with score=timestamp. Requests older than the window are removed. The count of remaining members determines if

the request is allowed. Atomic via Lua script.

RateLimiter.java — Sliding Window (Lua Script = Atomic)

```
@Component
public class RateLimiter {
    private final StringRedisTemplate redis;
    private static final int LIMIT = 100; // 100 requests
    private static final int WINDOW = 60; // per 60 seconds
    private static final String SCRIPT =
        "local key = KEYS[1]\n" +
        "local now = tonumber(ARGV[1])\n" +
        "local window = tonumber(ARGV[2])\n" +
        "local limit = tonumber(ARGV[3])\n" +
        "redis.call('ZREMRANGEBYSCORE', key, 0, now - window * 1000)\n" +
        "local count = redis.call('ZCARD', key)\n" +
        "if count < limit then\n" +
        "  redis.call('ZADD', key, now, now .. ':' .. math.random())\n" +
        "  redis.call('EXPIRE', key, window + 1)\n" +
        "  return 1\n" +
        "end\n" +
        "return 0";

    public boolean isAllowed(String ip, String endpoint) {
        String key = "shopflow:rate:" + ip + ":" + endpoint;
        long now = System.currentTimeMillis();
        Long result = redis.execute(new DefaultRedisScript<>(SCRIPT, Long.class),
            List.of(key), String.valueOf(now), String.valueOf(WINDOW),
            String.valueOf(LIMIT));
        return Long.valueOf(1L).equals(result);
    }
}
```

Algorithm	Data Structure	Accuracy	Memory per IP	Atomic?
Fixed window	INCR + EXPIRE on minute key	100% burst at boundary	~100 bytes per endpoint	Yes (single INCR)
Sliding window (ShopFlow)	SORTED SET per IP+endpoint	Exact — no boundary bursts	~500 bytes per endpoint	Yes (Lua script)
Token bucket	HASH: tokens + lastRefill time	Exact with burst allowance	~120 bytes per endpoint	Yes (Lua script)
Leaky bucket	LIST as queue	Exact — constant drain rate	~1 byte × queue depth	No — requires MULTI/EXEC

■ Redis Lua scripts execute atomically on a single Redis node — no race conditions between ZREMRANGEBYSCORE, ZCARD, and ZADD. Without Lua, three separate commands could be interleaved by concurrent requests. EVAL runs the script server-side. Use SHA1 EVALSHA for production (cache the script hash — avoids sending 500-byte script on every request).

08-07

Cache Invalidation Strategies

Cache invalidation is notoriously difficult. ShopFlow uses a combination of TTL-based expiry, write-through invalidation on known updates, and event-driven invalidation via DynamoDB Streams for Aurora changes that do not pass through the product service.

Cache Invalidation Strategies — ShopFlow

TTL-based (passive expiry)

Product cache: TTL=3600s. Stale data max 1 hour. Sufficient for catalog.

Session cache: TTL=900s (= JWT expiry). Always correct.

Write-through (active invalidation on write)

ProductService.updateProduct() → @CacheEvict("products")

Guarantees: readers see updated product within 1 cache miss

Event-driven (async via SQS/Lambda)

Aurora → DMS Change Data Capture → SQS → Lambda → Redis DEL

Handles out-of-band DB writes (migrations, data fixes, admin tools)

Cache stampede prevention: probabilistic early expiry

Recompute cache at $TTL \times 0.75$ with 10% probability — prevents cliff

Cache Stampede Prevention — Probabilistic Early Recomputation

```
// Problem: TTL=3600s. At second 3600, 1000 concurrent requests all miss
// and all hit Aurora simultaneously — "thundering herd"
// Solution: XFetch algorithm — probabilistic early expiry
public ProductDto getProductWithXFetch(String productId) {
    String key = "shopflow:product:" + productId;
    CachedValue cached = redis.opsForValue().get(key + ":xfetch");
    if (cached != null) {
        long ttlRemaining = redis.getExpire(key, TimeUnit.SECONDS);
        double beta = 1.0;
        double delta = cached.getComputeTime(); // ms to compute value
        // Early recompute if: -ttlRemaining < delta * beta * log(random)
        boolean earlyRecompute = -ttlRemaining < delta * beta * Math.log(Math.random());
        if (!earlyRecompute) return cached.getValue();
    }
    // Recompute from Aurora and update cache
    ProductDto fresh = productRepo.findById(productId).map(ProductDto::fromEntity)
        .orElse(null);
    redis.opsForValue().set(key + ":xfetch", new CachedValue(fresh, 5L),
        Duration.ofSeconds(3600));
    return fresh;
}
```

■ For most production deployments, a simpler stampede prevention suffices: use a distributed lock (SET NX PX) to allow only one request to recompute on cache miss, while all concurrent requests wait on the lock or serve stale data with GETEX (get + update expiry). The XFetch algorithm is optimal but complex — profile first to confirm stampede is actually occurring.

08-08

Pub/Sub & Redis Streams for Event Fan-Out

Redis Pub/Sub enables fire-and-forget event broadcasting — price change events broadcast to all subscribed product-svc instances. Redis Streams provide durable, consumer-group-based event processing — ShopFlow uses Streams for the real-time order dashboard.

Price Change Pub/Sub — Broadcast Cache Invalidation

```
// Publisher (admin service — after product price update in Aurora)
@Service
public class PriceEventPublisher {
    private final RedisTemplate<String,String> redis;
    public void publishPriceChange(String productId, BigDecimal newPrice) {
        String message = productId + ":" + newPrice.toPlainString();
        redis.convertAndSend("shopflow:price-updates", message);
    }
}

// Subscriber (all product-svc ECS instances — each receives ALL messages)
@Component
public class PriceCacheInvalidator implements MessageListener {
    private final RedisTemplate<String,Object> redis;
    @Override
    public void onMessage(Message message, byte[] pattern) {
        String[] parts = new String(message.getBody()).split(":");
        String productId = parts[0];
        redis.delete("shopflow:product:" + productId); // invalidate cache
        redis.delete("shopflow:product:" + productId + ":xfetch");
        log.info("Cache invalidated for product {}", productId);
    }
}
```

Feature	Pub/Sub	Redis Streams
Durability	Not durable — missed if offline	Persistent — all messages retained (configurable)
Consumer groups	No — all subscribers receive all messages	Yes — competing consumers; XACK per message
Replay/backfill	No — past messages lost	Yes — XREAD from any offset or "0" for start
ShopFlow use	Price change invalidation broadcast	Order event fan-out: dashboard + analytics
Delivery guarantee	At-most-once	At-least-once (with XACK consumer groups)

■ Redis Pub/Sub messages are not persistent — a subscriber that is disconnected (ECS task restart, deployment) will miss all messages during downtime. NEVER use Pub/Sub for critical business events (order placed, payment captured). Use SQS, Kinesis, or Redis Streams for events that must not be lost. Use Pub/Sub only for cache invalidation where TTL provides eventual correctness even if broadcasts are missed.

08-09

Eviction Policies & Memory Management

When Redis reaches maxmemory, it evicts keys based on the configured policy. ShopFlow uses allkeys-lru — most recently used keys are retained, least recently used are evicted regardless of whether they have TTLs. This maximizes cache hit rate by keeping hot keys in memory.

Eviction Policy	Behavior	Best For	ShopFlow?
noeviction	Return error on write when full	Session stores where loss = data loss	No — would crash product-svc on cache write
allkeys-lru	Evict least-recently-used across all keys	General-purpose cache where all keys are expendable	YES — expirable cache, session cache
volatile-lru	LRU eviction only among keys with TTL	Mixed workload: some keys must never be evicted	Never-evicted complexity
allkeys-lfu	Evict least-frequently-used (new Redis 6.0)	Read-heavy access: protect highly accessed keys from backfill	Consider for Backfill RU — product page history
allkeys-random	Random eviction	Testing only	No
volatile-ttl	Evict key with shortest TTL remaining	TTL-based priority: soonest-to-expire first	Never — can evict fresh keys with short TTL

Memory Optimization Techniques

```
# Redis memory usage for ShopFlow product cache
# 400K products × avg key+value size:
```

```
# Key: "shopflow:product:{20-char-UUID}" = ~50 bytes × 400K = 20MB
# Value (HASH): name(40B)+price(8B)+stock(4B)+category(15B) = ~100 bytes × 400K = 40MB
# Redis overhead per key: ~80 bytes × 400K = 32MB
# Total: 20 + 40 + 32 = 92MB — well within 26GB r7g.large
# Memory optimization: use HASH instead of STRING for structured objects
# STRING with JSON: {name:"...",price:29.99,...} serialized = ~200B per product
# HASH with fields: HSET key name "..." price "29.99" = ~100B per product (50% less)
# Check memory fragmentation ratio (should be 1.0-1.5):
redis-cli INFO memory | grep mem_fragmentation_ratio
# > 1.5: too much fragmentation → schedule MEMORY PURGE
# < 1.0: Redis swapping to disk → increase node size immediately
```

■ ElastiCache Redis 7 supports Data Tiering (r6gd family with NVMe SSD): items not recently accessed are tiered to NVMe, keeping hot items in DRAM. For ShopFlow product cache with 400K products but 80% of requests going to top 5% (20K) products: data tiering reduces DRAM cost by 50% while hot items remain <1ms. Not used in ShopFlow currently as the full 8GB fits in r7g.large DRAM.

08-10

High Availability — Failover & Cluster Topology

ElastiCache Multi-AZ replication group: primary in us-east-1a, replicas in us-east-1b and us-east-1c. On primary failure, ElastiCache promotes a replica to primary in under 60 seconds. Spring's Lettuce client reconnects automatically; requests during failover see <60s of elevated latency.

ShopFlow Redis Topology — Multi-AZ Replication Group

Primary node: cache.r7g.large (us-east-1a)

All writes: SET, HSET, DEL, EXPIRE, ZADD, INCR

Sync replication to replica-1 and replica-2

Replica-1: cache.r7g.large (us-east-1b)

Read-From-REPLICA_PREFERRED: session + product GETs routed here

Replica-2: cache.r7g.large (us-east-1c)

Read overflow and standby for failover promotion

Failover sequence (primary failure detected)

1. ElastiCache detects failure via health check (~10s)

2. Promotes replica-1 to primary (~5s)

3. DNS for primary endpoint updated (~5s)

4. Lettuce reconnects via DNS change (~15s with topology refresh)

Lettuce Topology Refresh for Failover Detection

```
// Enable adaptive topology refresh so Lettuce detects failover quickly
ClusterTopologyRefreshOptions topologyRefreshOptions =
ClusterTopologyRefreshOptions.builder()
.enablePeriodicRefresh(Duration.ofMinutes(5))
.enableAdaptiveRefreshTrigger( // react on errors
RefreshTrigger.MOVED_REDIRECT, // MOVED = wrong node
RefreshTrigger.PERSISTENT_RECONNECTS) // reconnect = failover
.adaptiveRefreshTriggersTimeout(Duration.ofSeconds(30))
```

```

.build();
ClusterClientOptions clientOptions = ClusterClientOptions.builder()
.topologyRefreshOptions(topologyRefreshOptions)
.validateClusterNodeMembership(false) // tolerate stale topology during failover
.build();

```

HA Configuration	Failover Time	Data Loss on Failover	Monthly Cost
Single node (no replicas)	Manual — minutes to hours	All data lost	\$0.157/hr × 720 = \$113/mo
Replication group, no Multi-AZ	~5 min (manual failover)	Possible replica lag loss	\$0.157/hr × 3 = \$340/mo
Multi-AZ replication (ShopFlow)	<60s automatic failover	Replica lag (typically <1s)	\$0.157/hr × 3 = \$340/mo + \$0/mo (Mu

■ Configure Lettuce's adaptive topology refresh — without it, Lettuce caches the primary endpoint DNS and will keep sending writes to the old primary IP after failover until the next periodic refresh (default: 10 minutes). With adaptive refresh triggered by PERSISTENT_RECONNECTS, Lettuce detects the reconnection after failover and refreshes topology in 15-30 seconds.

08-11

Security — TLS, AUTH, and Network Isolation

ElastiCache Redis security: Transit Encryption (TLS 1.2) encrypts data in transit between ECS tasks and Redis, At-Rest Encryption (AES-256) protects snapshots, AUTH token requires password authentication, and Security Groups restrict access to the app tier only.

Security Control	Configuration	ShopFlow Setup	Threat Mitigated
Transit Encryption (TLS)	transitEncryptionEnabled=true	TLS 1.2+; enforced in Spring client	Man-in-the-middle on VPC traffic
AUTH token	authToken from Secrets Manager	64-char random token; rotated 90 days	Unauthorized Redis access if SG misconfigured
At-rest encryption	atRestEncryptionEnabled=true	AES-256 for snapshots + replication	Snapshot access if S3 bucket exposed
Security Group	Port 6379; source=sgEcs only	App-tier SG; no direct access from public	Unauthorized access from non-app workloads
No public access	Subnet: PRIVATE_ISOLATED	No route to internet gateway	External Redis access attempts
VPC isolation	Same VPC as ECS (10.0.0.0/24)	App tier: 10.0.3.0/24, 10.0.4.0/24, 10.0.5.0/24	Cross-account access

Secrets Manager Rotation for Redis AUTH Token

```

// CDK: create secret and enable 90-day rotation
const redisAuthSecret = new secretsmanager.Secret(this, "RedisAuthToken", {
secretName: "/shopflow/redis/auth-token",
generateSecretString: {
passwordLength: 64,
excludePunctuation: true, // Redis AUTH does not accept all special chars
},
});
// For AUTH token rotation: Lambda rotator must update ElastiCache + Secrets Manager
// ElastiCache supports dual-token during rotation window:
// 1. Add new token (ElastiCache allows 2 active tokens during rotation)
// 2. Update Spring app env var REDIS_AUTH_TOKEN → rolling ECS deploy
// 3. Remove old token after all ECS tasks are on new token

```

■ ElastiCache supports having two AUTH tokens simultaneously during the rotation window. The rotation Lambda adds the new token while the old one remains valid — ECS tasks continue to authenticate with the old token until they pick up the new one on rolling redeploy. Only remove the old token after all tasks have completed the rolling update. Verify via CloudWatch Metrics that AuthenticationFailures = 0 before removing old token.

08-12

@Cacheable, @CacheEvict & Spring Cache Config

Spring Cache abstraction provides declarative caching via @Cacheable, @CachePut, and @CacheEvict. ShopFlow configures RedisCacheManager with per-cache TTLs, JSON serialization, and null-value caching prevention. This

reduces boilerplate vs manual RedisTemplate calls.

RedisCacheConfig.java — Per-Cache TTL Config

```
@Configuration
@EnableCaching
public class RedisCacheConfig {
    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory cf) {
        RedisCacheConfiguration defaults = RedisCacheConfiguration.defaultCacheConfig()
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()))
            .disableCachingNullValues(); // prevent null poison entries
        Map<String, RedisCacheConfiguration> cacheConfigs = Map.of(
            "products", defaults.entryTtl(Duration.ofHours(1)),
            "categories", defaults.entryTtl(Duration.ofHours(6)),
            "homeFeed", defaults.entryTtl(Duration.ofMinutes(5)),
            "searchSuggestions", defaults.entryTtl(Duration.ofMinutes(30)),
            "userProfiles", defaults.entryTtl(Duration.ofMinutes(30))
        );
        return RedisCacheManager.builder(cf)
            .cacheDefaults(defaults.entryTtl(Duration.ofMinutes(10)))
            .withInitialCacheConfigurations(cacheConfigs)
            .build();
    }
}
```

Service Annotations

```
@Cacheable(value="products",key="#productId",unless="#result==null")
public ProductDto getProduct(String productId) { ... }

@CacheEvict(value="products",key="#dto.productId")
public ProductDto updateProduct(ProductDto dto) { ... }

@CachePut(value="products",key="#result.productId")
public ProductDto createProduct(ProductDto dto) { ... } // populate on create

// Evict multiple caches on product delete
@Caching(evict={
    @CacheEvict(value="products",key="#productId"),
    @CacheEvict(value="homeFeed",allEntries=true), // invalidate all feed pages
    @CacheEvict(value="categories",allEntries=true) // refresh category listings
})
public void deleteProduct(String productId) { ... }
```

■ Spring `@Cacheable` on a method called from within the same class bypasses the cache — Spring AOP creates a proxy, so internal calls skip the proxy. Move `@Cacheable` methods to a separate `@Service` class or inject the service into itself (self-injection). Alternatively, use AspectJ weaving instead of Spring proxy-based AOP to enable internal call interception.

08-13

Redis Cluster Mode — When to Scale to Multiple Shards

Elasticache cluster-mode enabled splits the keyspace across multiple shards (each shard = 1 primary + replicas). ShopFlow uses cluster-mode disabled (single shard) since the full 8GB dataset fits in one r7g.large (26GB). Cluster mode is needed when write throughput exceeds primary capacity or dataset exceeds node RAM.

Criteria	Single Shard (ShopFlow)	Cluster Mode (Multiple Shards)
Dataset size	8 GB (< 26 GB node RAM)	> node RAM — distribute across shards
Write throughput	< 100,000 ops/s	> 100,000 writes/s — single primary saturated
Horizontal write scaling	Not possible — single primary	Yes — each shard has own primary
Multi-key commands (MGET, MSET)	Any keys	Keys must be in same shard (hash slot)

Criteria	Single Shard (ShopFlow)	Cluster Mode (Multiple Shards)
Lua scripts	Any keys	All keys must be in same shard
Resharding (rebalance)	N/A — single shard	Online resharding — move hash slots without downtime
Cost	3 nodes (1P + 2R)	6 nodes (2 shards × 1P + 2R) — 2× cost

Hash Tags for Multi-Key Operations in Cluster Mode

```
// Problem: cluster mode hashes key to determine shard
// MGET "shopflow:product:p-1" "shopflow:product:p-2"
// → p-1 on shard 0, p-2 on shard 1 → CROSSSLOT error
// Solution: hash tags — only the part in {} is hashed
// "shopflow:product:{category-electronics}:p-1"
// "shopflow:product:{category-electronics}:p-2"
// Both hash on "category-electronics" → same shard → MGET works
// ShopFlow grouping strategy: hash on userId for user-specific keys
// "shopflow:session:{u-550e8400}:token"
// "shopflow:cart:summary:{u-550e8400}"
// Both on same shard — one MGET fetches session + cart for auth middleware
```

■ Upgrade path from single-shard to cluster mode: (1) Create new cluster-mode enabled replication group with 2 shards; (2) Use SCAN + DUMP/RESTORE to migrate keys; (3) Test multi-key commands — all must use hash tags; (4) Update Lettuce config to ClusterClientConfiguration. Lettuce cluster-mode support is transparent for single-key commands but requires hash tag discipline for MGET and Lua scripts.

08-14

Monitoring, Alarms & Performance Tuning

Key ElastiCache metrics: CacheHits, CacheMisses, CacheHitRate, Evictions, CurrConnections, BytesUsedForCache, and EngineCPUUtilization. ShopFlow alarms on hit rate dropping below 90% (product cache issue) and Evictions above 0 (memory pressure).

Metric	Namespace	Alert Threshold	Root Cause & Action
CacheHitRate	AWS/ElastiCache	< 90% for >5 min	Cold cache after deploy; items evicted; TTL too short
Evictions	AWS/ElastiCache	> 100/min sustained	Memory full — increase node size or reduce TTLs. C
CurrConnections	AWS/ElastiCache	> 500 per node	Connection leak in app; maxPoolSize too high. Chec
ReplicationLag	AWS/ElastiCache	> 500ms	Heavy primary write load; replica under-sized. Check
EngineCPUUtilization	AWS/ElastiCache	> 80%	Complex Lua scripts or SCAN/KEYS commands. Av
NetworkBandwidthOutAllowance	AWS/ElastiCache	Any	Cache.r7g.large: 12.5 Gbps. Upgrade node if sustain

CloudWatch Alarm CDK

```
new cloudwatch.Alarm(this, "RedisLowHitRate", {
  metric: new cloudwatch.Metric({
    namespace: "AWS/ElastiCache",
    metricName: "CacheHitRate",
    dimensionsMap: {
      CacheClusterId: redis.ref,
    },
    statistic: "Average",
    period: Duration.minutes(5),
  }),
  threshold: 0.90, // alert when hit rate drops below 90%
  comparisonOperator: cloudwatch.ComparisonOperator.LESS_THAN_THRESHOLD,
  evaluationPeriods: 3,
  datapointsToAlarm: 2,
}).addAlarmAction(new cwa.SnsAction(opsAlertTopic));
```

Performance Tuning Checklist

Tuning Action	Command	Target
Identify large keys (slow)	redis-cli --bigkeys	No key >1MB; HASH >1000 fields
Find slowest commands	redis-cli SLOWLOG GET 25	p99 <10ms per command
Check memory fragmentation	INFO memory grep mem_fragmentation_ratio	1.0–1.5
Check keyspace hit ratio	INFO stats grep keyspace	Hits/(hits+misses) > 0.90
Connection count	INFO clients grep connected_clients	< 500 per node

■ NEVER run KEYS * in production Redis. KEYS is O(N) across the entire keyspace — at 50M cached objects, KEYS * blocks Redis for several seconds, causing all application requests to time out. Use SCAN with COUNT 100 (cursor-based, non-blocking) for key enumeration. Add this rule to team runbook and enforce with Redis ACL NOCOMMANDS KEYS.

08-15

Cost Optimization — Reserved Nodes & Right-Sizing

ElastiCache costs: on-demand cache.r7g.large = \$0.228/hr (\$164/mo per node). ShopFlow runs 3 nodes = \$492/mo. Reserved Node (1-year, no-upfront) = \$0.143/hr = \$103/mo per node = \$309/mo total. 1-year reserved saves 37% (\$183/mo). 3-year reserved saves 55%.

Cost Component	On-Demand	1-yr Reserved (All-Upfront)	3-yr Reserved (All-Upfront)
cache.r7g.large (per node)	\$0.228/hr = \$164/mo	\$0.102/hr = \$73/mo (55% off)	\$0.075/hr = \$54/mo (67% off)
3-node replication group	\$492/mo	\$219/mo	\$162/mo
Annual (3 nodes)	\$5,904	\$2,628 → save \$3,276/yr	\$1,944 → save \$3,960/yr
Backup storage (<cluster size)	Free	Free	Free
Data transfer (same region)	Free (within VPC)	Free	Free

Right-Sizing Decision Tree

```
# Step 1: Measure current memory usage
redis-cli INFO memory | grep used_memory_human
# Example: used_memory_human:8.23G

# Step 2: Check Evictions metric in CloudWatch
# Evictions = 0: node is correctly sized or over-sized
# Evictions > 0: under-sized – increase node or reduce TTLs

# Step 3: Check EngineCPUUtilization
# < 30%: node is CPU over-provisioned for workload
# > 80%: CPU bottleneck – scale up node or use Lua script less

# Step 4: Check CurrConnections
# cache.r7g.large max connections: 65,536
# ShopFlow: 20 ECS tasks × 20 pool connections = 400 connections (0.6% used)

# Step 5: Right-size recommendation
# Used: 8GB / Available: 26GB = 31% → Headroom for Black Friday spike to 2x
# Decision: r7g.large is correct. Do NOT downsize to m7g.large (6.4GB RAM).
```

Node Type	RAM	Max Connections	Price/hr (on-demand)	ShopFlow Fit
cache.m7g.medium	3.09 GB	1,500	\$0.065	Too small — 8GB dataset exceeds RAM
cache.m7g.large	6.38 GB	1,500	\$0.137	Too small — no headroom for BF spike
cache.r7g.large (ShopFlow)	26.32 GB	1,500	\$0.228	Correct — 8GB used, 2× headroom
cache.r7g.xlarge	52.82 GB	1,500	\$0.452	Oversized — 8GB in 52GB = 15% utilization

■ ElastiCache Serverless (2024): pay per ECU (ElastiCache Compute Unit) and per-GB storage — no node provisioning. Scales 0 to millions of ops/s instantly. Cost: \$0.00034 per ECU-second + \$0.125/GB-hour. Break-even vs r7g.large cluster

(~3 nodes): around 500 ECU sustained. For ShopFlow dev/staging with intermittent load, Serverless reduces cost by 70%. Production: reserved r7g.large cluster is more cost-predictable.

08-BP

Best Practices & Anti-Patterns

- ✓ Use allkeys-lru eviction policy for all-cache workloads — prevents OOM errors on memory pressure
- ✓ Set TTL on every key — unbounded cache growth leads to memory exhaustion and unexpected evictions
- ✓ Use HASH type for structured objects — 50% lower memory vs JSON STRING; field-level HGET operations
- ✓ Implement circuit breaker for Redis calls — cache must be optional, not required for correctness
- ✓ Use Lua scripts for multi-step atomic operations — ZADD+ZCARD+ZREMRANGEBYSCORE in one round trip
- ✓ Hash the token when using JWT as Redis key — SHA-256 hash prevents raw token exposure in redis-cli logs
- ✓ Configure Lettuce adaptive topology refresh — detect failover in <30s instead of default 10 min
- ✓ Enable Multi-AZ with automatic failover — promotes replica in <60s on primary failure
- ✓ Use ReadFrom.REPLICA_PREFERRED for reads — reduce primary load; replicas handle 80%+ of ShopFlow reads
- ✓ Enable transit encryption (TLS) and AUTH token — defense in depth even within private VPC
- ✓ Monitor CacheHitRate and alert below 90% — early warning of cache effectiveness degradation
- ✓ Use SCAN with COUNT for key enumeration in scripts — never KEYS * in production
- ✓ Compress large cached values (GZIP) before SET — reduce memory usage and network bandwidth
- ✓ Use disableCachingNullValues() in Spring RedisCacheManager — prevents null poison entries
- ✓ Size Redis node with 2× working set headroom — r7g.large at 8GB used of 26GB is correct headroom
- ✓ Reserve ElastiCache nodes 1-year upfront for production — 55% savings vs on-demand for steady workload
- Never use KEYS * in production — O(N) scan blocks Redis for seconds; causes application-wide timeouts
- Do not rely on Redis for durability — Redis is a cache; Aurora is the source of truth. Never write to Redis only
- Avoid large items (>100KB) in Redis — serialization overhead kills throughput; store in S3, cache reference key
- Do not use connection-per-request pattern — always use Lettuce connection pool; connections are expensive
- Do not use Pub/Sub for critical events — missed if subscriber is offline; use SQS/Kinesis instead
- Avoid Sorted Set with millions of members per key — ZRANGE with LIMIT is O(log N + M); huge sets degrade performance
- Do not disable AUTH token in production — even in private VPC, lateral movement from compromised ECS task can reach Redis

08-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
allkeys-lru policy	Evicts least-recently-used key across ALL keys when maxmemory reached. Best for pure cache workloads.
HASH vs STRING for objects	HASH: O(1) field access, 50% lower memory. STRING(JSON): simpler, requires full object deserialization.
Redis Pub/Sub durability	Not durable — subscribers offline miss all messages. Use only for fire-and-forget (cache invalidation).
Lettuce vs Jedis	Lettuce: non-blocking NIO, thread-safe single connection (or pool). Jedis: blocking, one connection per thread.
Multi-AZ failover time	ElastiCache promotes replica to primary in <60s. DNS propagates in ~15s. Lettuce adaptive refresh detects failover.
Cache stampede	All concurrent cache misses hit DB simultaneously on TTL expiry. Prevention: probabilistic early expiry.
TTL best practices	Set TTL on every key. TTL should be shorter than acceptable staleness. Session: TTL=JWT expiry (15 min).
SCAN vs KEYS	KEYS O(N): blocks all Redis ops during scan. SCAN with COUNT: cursor-based, returns ~COUNT keys.
Reserved vs on-demand	1-year Reserved (No-Upfront): ~40% off on-demand. All-Upfront: ~55% off. Use Reserved for production.
ElastiCache Serverless	Pay-per-use ECU + GB. Auto-scales 0-unlimited. No node management. Best for dev/test, variable workloads.

Concept	Key Value / Rule
ReadFrom.REPLICA_PREFERRED	Reads routed to replica (eventual consistency). Falls back to primary if no replica available. Use for all reads.
Cache hit rate formula	$\text{CacheHits} / (\text{CacheHits} + \text{CacheMisses})$. Target: >90% for product cache. Below 80%: TTL too short.

Top Interview Questions

Question	Answer
Which Redis data structure for Sorted Set (ZADD score=timestamp). ZREMRANGEBYSCORE removes old requests. ZCARD counts current elements.	Sorted Set (ZADD score=timestamp). ZREMRANGEBYSCORE removes old requests. ZCARD counts current elements.
How to prevent cache stampede in Redis?	Three Approaches: (1) Probabilistic early recomputation (XFetch): recompute at 75% of TTL with probability 0.5. (2) Mutex. (3) Distributed locks.
Cache-aside vs write-through Cache-aside (lazy): write DB, evict/invalidate cache. Reads: check cache first; miss → DB → populate. Better for read-heavy.	Cache-aside (lazy): write DB, evict/invalidate cache. Reads: check cache first; miss → DB → populate. Better for read-heavy.
Explain Redis Streams vs Pub/Sub	Streams: ordered, persistent log. Pub/Sub: fire-and-forget broadcast. No persistence. Missed if offline. Use for cache invalidation where TTL is short.
How to right-size ElastiCache	Step 1: Measure used_memory_human (redis INFO). Step 2: Evictions in CloudWatch (Evictions >0 = too small).